



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERU
FACULTAD DE INGENIERIA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS



Práctica SEMANA 13

Asignatura: Desarrollo de Aplicaciones Web (IS093A)

Unidad II: Desarrollo Web FullStack

Tema: Diseño de APIs RESTful con Django REST Framework (DRF): Serialización, ViewSets, Routers, Paginación, Filtrado, Throttling, CORS/CSRF e HATEOAS

Modalidad: Presencial / Laboratorio

Fecha: 02.07.2026

APELLIDOS Y NOMBRES: Barja Ortiz Erick Gerson

OBJETIVO DE LA PRÁCTICA

Diseñar, implementar y documentar una API RESTful production-ready utilizando Django REST Framework (DRF), aplicando los principios arquitectónicos REST (stateless, cacheable, layered system, uniform interface, code on demand, clientserver) y el estilo HATEOAS básico. Implementar serialización avanzada de modelos, configurar ViewSet + Router para routing automático, aplicar paginación estándar, filtrado por query parameters, y límites de tasa (throttling) para protección contra abuso. Configurar middleware de CORS y gestionar CSRF en entornos SPA/AJAX. Validar endpoints con herramientas HTTP (Postman/cURL), generar documentación OpenAPI/Swagger automática, y preparar la arquitectura para desacoplamiento y contenedores (Semana 14: Microservicios).

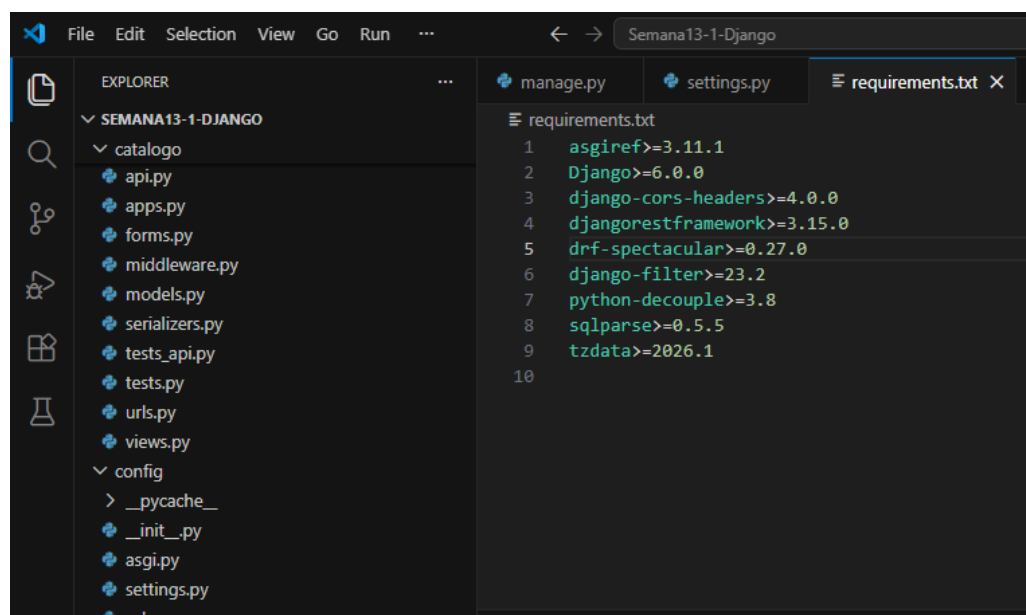
RECURSOS REQUERIDOS

- Python 3.11+ + Django 5.x + djangorestframework + django-cors-headers + drf-spectacular
- Visual Studio Code (Extensiones: Python, REST Client, Thunder Client / Postman, OpenAPI Viewer) • Terminal integrada + entorno virtual (venv)
- Postman, Insomnia o curl para pruebas HTTP
- Navegador + DevTools (Network, Headers, CORS preflight)
- Documentación oficial: django-rest-framework.org, django-corsheaders.readthedocs.io, drf-spectacular.readthedocs.io

- GitHub (control de versiones)

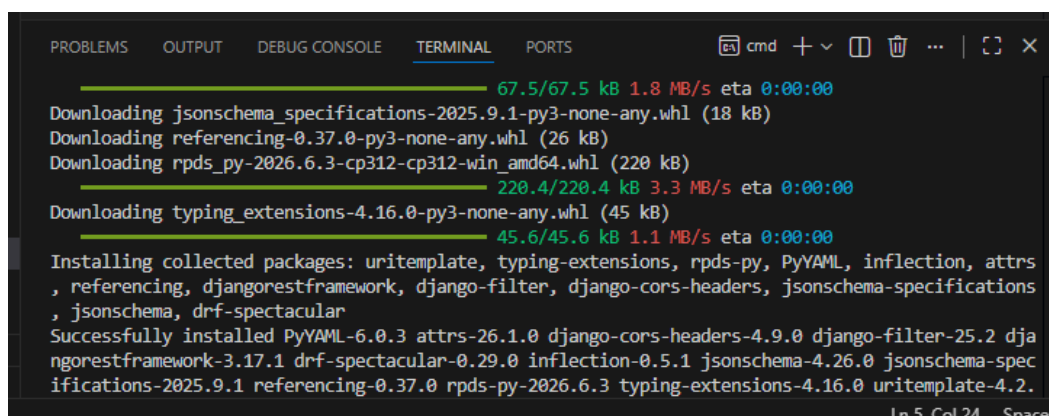
EJERCICIO PASO A PASO

1. Setup & Configuración DRF: Instalar `django-rest-framework`, `django-cors-headers`, `drf-spectacular`. Configurar `INSTALLED_APPS`, `REST_FRAMEWORK` (default pagination, throttling, auth), `CORS_ALLOWED_ORIGINS`, `SPECTACULAR_SETTINGS`. Verificar python manage.py check.



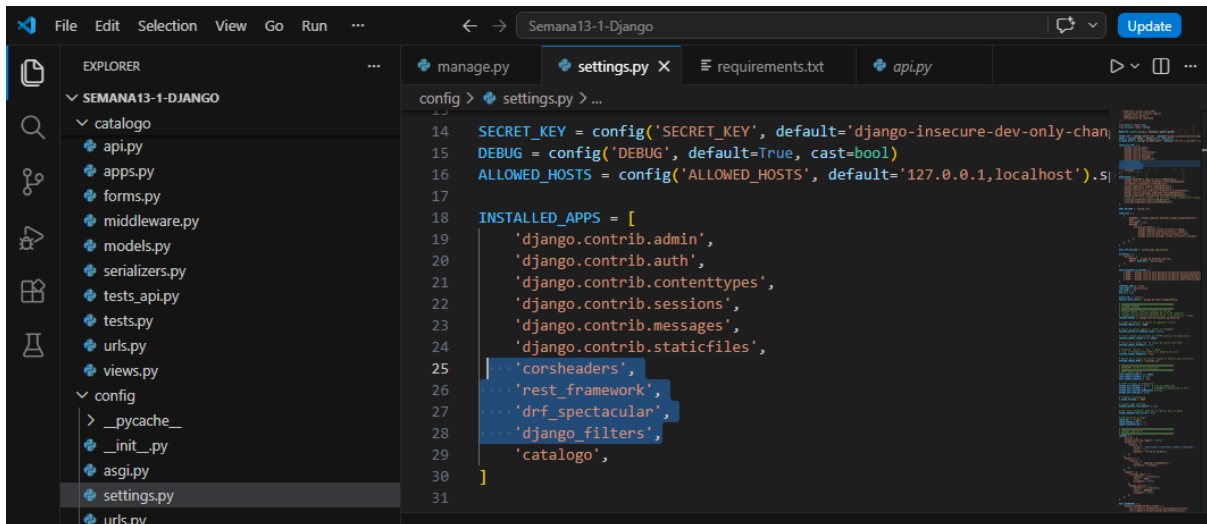
The screenshot shows the Visual Studio Code interface. On the left, the Explorer pane displays the project structure for 'SEMANA13-1-DJANGO'. The 'catalogo' folder is expanded, showing files like `api.py`, `apps.py`, `forms.py`, `middleware.py`, `models.py`, `serializers.py`, `tests_api.py`, `tests.py`, `urls.py`, and `views.py`. The 'config' folder is also expanded, showing `__pycache__`, `__init__.py`, `asgi.py`, `settings.py`, and `urls.py`. The main editor area shows the `requirements.txt` file with the following content:

```
1 asgiref>=3.11.1
2 Django>=6.0.0
3 django-cors-headers>=4.0.0
4 django-rest-framework>=3.15.0
5 drf-spectacular>=0.27.0
6 django-filter>=23.2
7 python-decouple>=3.8
8 sqlparse>=0.5.5
9 tzdata>=2026.1
10
```



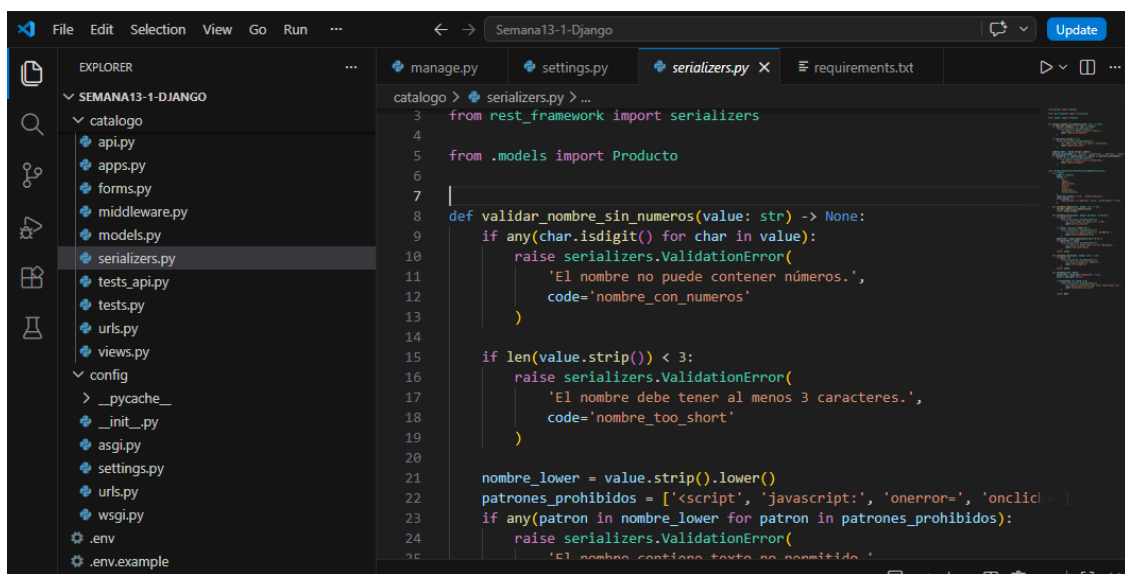
The screenshot shows the terminal window in VS Code. It displays the output of a command to install dependencies. The terminal shows the following progress:

```
67.5/67.5 kB 1.8 MB/s eta 0:00:00
Downloading jsonschema_specifications-2025.9.1-py3-none-any.whl (18 kB)
Downloading referencing-0.37.0-py3-none-any.whl (26 kB)
Downloading rpds_py-2026.6.3-cp312-cp312-win_amd64.whl (220 kB)
220.4/220.4 kB 3.3 MB/s eta 0:00:00
Downloading typing_extensions-4.16.0-py3-none-any.whl (45 kB)
45.6/45.6 kB 1.1 MB/s eta 0:00:00
Installing collected packages: uritemplate, typing-extensions, rpds-py, PyYAML, inflection, attrs, referencing, django-rest-framework, django-filter, django-cors-headers, jsonschema-specifications, jsonschema, drf-spectacular
Successfully installed PyYAML-6.0.3 attrs-26.1.0 django-cors-headers-4.9.0 django-filter-25.2 django-rest-framework-3.17.1 drf-spectacular-0.29.0 inflection-0.5.1 jsonschema-4.26.0 jsonschema-specifications-2025.9.1 referencing-0.37.0 rpds-py-2026.6.3 typing-extensions-4.16.0 uritemplate-4.2.
```

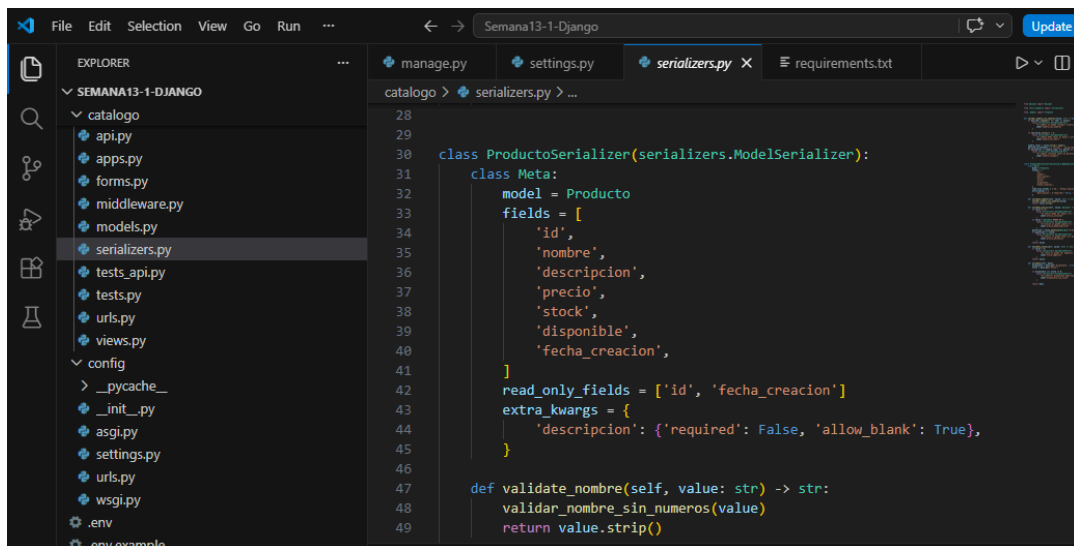


```
14 SECRET_KEY = config('SECRET_KEY', default='django-insecure-dev-only-cha
15 DEBUG = config('DEBUG', default=True, cast=bool)
16 ALLOWED_HOSTS = config('ALLOWED_HOSTS', default='127.0.0.1,localhost').s
17
18 INSTALLED_APPS = [
19     'django.contrib.admin',
20     'django.contrib.auth',
21     'django.contrib.contenttypes',
22     'django.contrib.sessions',
23     'django.contrib.messages',
24     'django.contrib.staticfiles',
25     'corsheaders',
26     'rest_framework',
27     'drf_spectacular',
28     'django_filters',
29     'catalogo',
30 ]
```

2. Serialización Avanzada: Crear serializers.py. Implementar ModelSerializer, campos read-only (ReadOnlyField, SerializerMethodField), validación anidada (validate()), y representación hipervinculada (HyperlinkedModelSerializer).



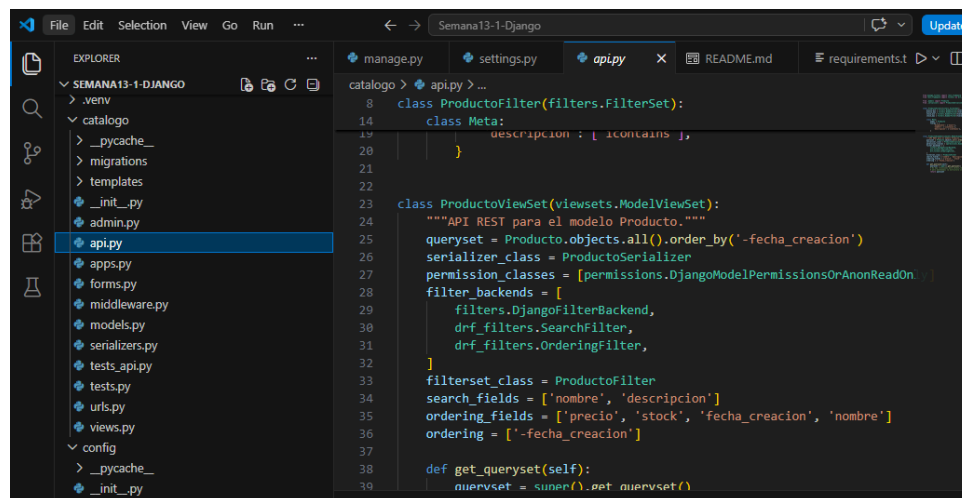
```
3 from rest_framework import serializers
4
5 from .models import Producto
6
7
8 def validar_nombre_sin_numeros(value: str) -> None:
9     if any(char.isdigit() for char in value):
10         raise serializers.ValidationError(
11             'El nombre no puede contener números.',
12             code='nombre_con_numeros'
13         )
14
15     if len(value.strip()) < 3:
16         raise serializers.ValidationError(
17             'El nombre debe tener al menos 3 caracteres.',
18             code='nombre_too_short'
19         )
20
21     nombre_lower = value.strip().lower()
22     patrones_prohibidos = ['<script', 'javascript:', 'onerror=', 'onclick=']
23     if any(patron in nombre_lower for patron in patrones_prohibidos):
24         raise serializers.ValidationError(
25             'El nombre contiene texto no permitido.'
```



The screenshot shows the Visual Studio Code editor with a Django project named 'Semana13-1-Django'. The Explorer sidebar on the left shows the project structure, with 'serializers.py' selected under the 'catalogo' directory. The main editor window displays the code for 'ProductoSerializer' in 'serializers.py'.

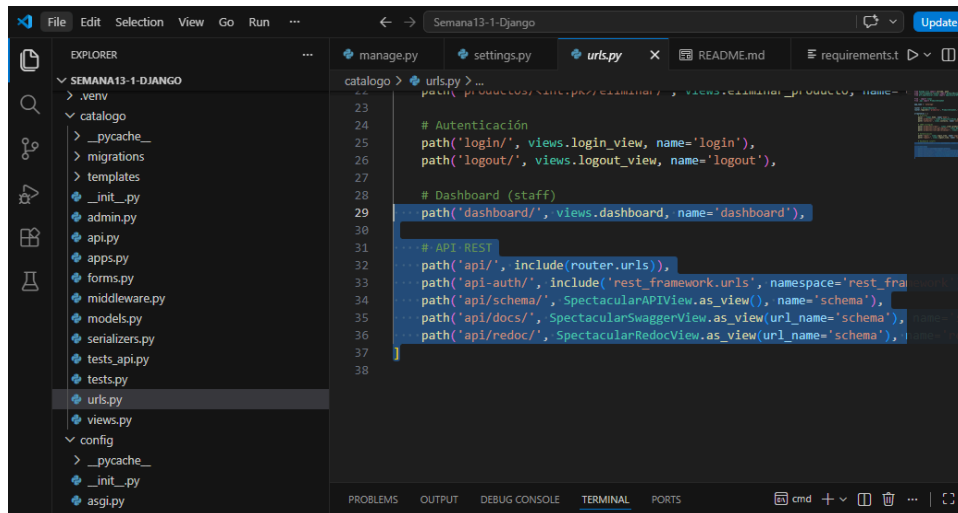
```
28
29
30 class ProductoSerializer(serializers.ModelSerializer):
31     class Meta:
32         model = Producto
33         fields = [
34             'id',
35             'nombre',
36             'descripcion',
37             'precio',
38             'stock',
39             'disponible',
40             'fecha_creacion',
41         ]
42         read_only_fields = ['id', 'fecha_creacion']
43         extra_kwargs = {
44             'descripcion': {'required': False, 'allow_blank': True},
45         }
46
47 def validate_nombre(self, value: str) -> str:
48     validar_nombre_sin_numeros(value)
49     return value.strip()
```

3. ViewSets & Routers: Convertir vistas estándar a ModelViewSet. Configurar queryset, serializer_class, permission_classes, filter_backends, pagination_class. Registrar en DefaultRouter o SimpleRouter. Incluir en urls.py.

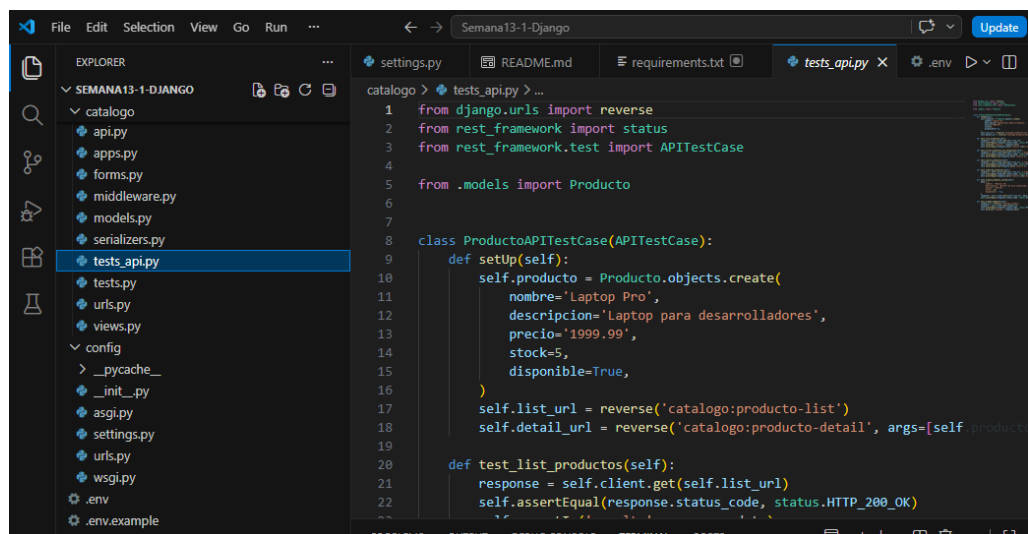


The screenshot shows the Visual Studio Code editor with the same Django project. The Explorer sidebar shows 'api.py' selected under the 'catalogo' directory. The main editor window displays the code for 'ProductoFilter' and 'ProductoViewSet' in 'api.py'.

```
8 class ProductoFilter(filters.FilterSet):
14     class Meta:
15         description = ['contains'],
16         fields = ['nombre', 'descripcion', 'precio', 'stock', 'disponible', 'fecha_creacion']
17
18
19
20
21
22
23 class ProductoViewSet(viewsets.ModelViewSet):
24     """API REST para el modelo Producto."""
25     queryset = Producto.objects.all().order_by('-fecha_creacion')
26     serializer_class = ProductoSerializer
27     permission_classes = [permissions.DjangoModelPermissionsOrAnonReadOnly]
28     filter_backends = [
29         filters.DjangoFilterBackend,
30         drf_filters.SearchFilter,
31         drf_filters.OrderingFilter,
32     ]
33     filterset_class = ProductoFilter
34     search_fields = ['nombre', 'descripcion']
35     ordering_fields = ['precio', 'stock', 'fecha_creacion', 'nombre']
36     ordering = ['-fecha_creacion']
37
38 def get_queryset(self):
39     queryset = super().get_queryset()
```

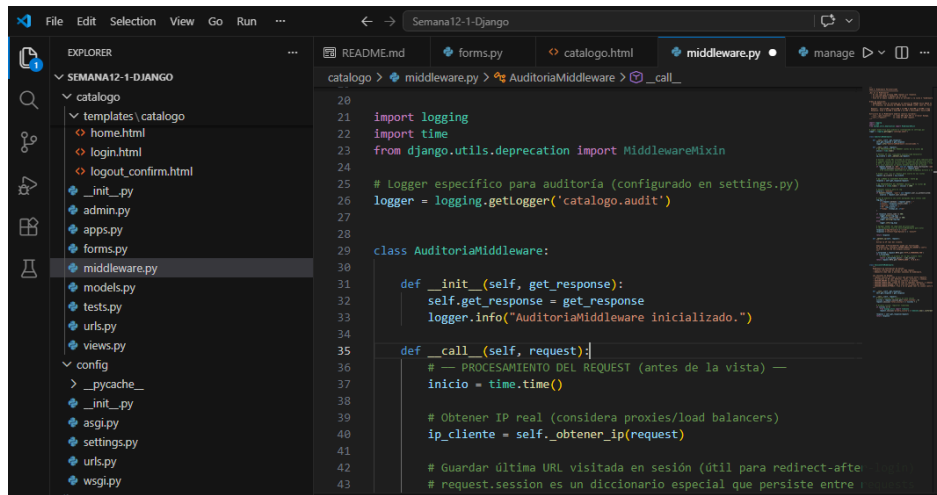


```
1 from django.urls import path, include
2
3 # Autenticación
4 path('login/', views.login_view, name='login'),
5 path('logout/', views.logout_view, name='logout'),
6
7 # Dashboard (staff)
8 path('dashboard/', views.dashboard, name='dashboard'),
9
10 # API REST
11 path('api/', include(router.urls)),
12 path('api-auth/', include('rest_framework.urls', namespace='rest_framework')),
13 path('api/schema/', SpectacularAPIView.as_view(), name='schema'),
14 path('api/docs/', SpectacularSwaggerView.as_view(url_name='schema'), name='swagger-ui'),
15 path('api/redoc/', SpectacularRedocView.as_view(url_name='schema'), name='redoc')
```



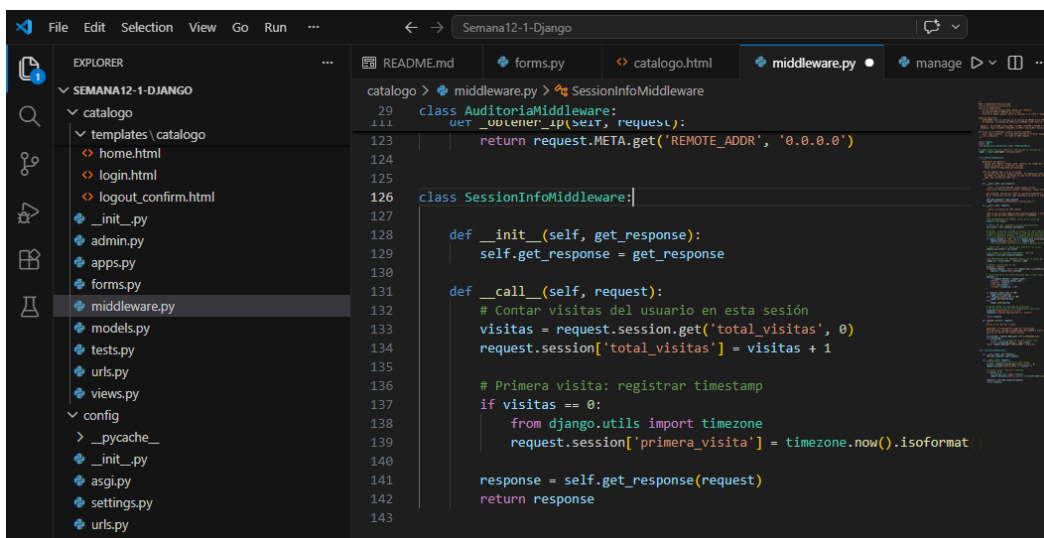
```
1 from django.urls import reverse
2 from rest_framework import status
3 from rest_framework.test import APITestCase
4
5 from .models import Producto
6
7
8 class ProductoAPITestCase(APITestCase):
9     def setUp(self):
10         self.producto = Producto.objects.create(
11             nombre='Laptop Pro',
12             descripcion='Laptop para desarrolladores',
13             precio='1999.99',
14             stock=5,
15             disponible=True,
16         )
17         self.list_url = reverse('catalogo:producto-list')
18         self.detail_url = reverse('catalogo:producto-detail', args=[self.producto.id])
19
20     def test_list_productos(self):
21         response = self.client.get(self.list_url)
22         self.assertEqual(response.status_code, status.HTTP_200_OK)
```

4. Filtrado, Paginación & Throttling: Configurar DjangoFilterBackend, SearchFilter, OrderingFilter. Aplicar PageNumberPagination o LimitOffsetPagination. Configurar AnonRateThrottle / UserRateThrottle. Probar con query params (?status=activo&page=2&ordering=-created).



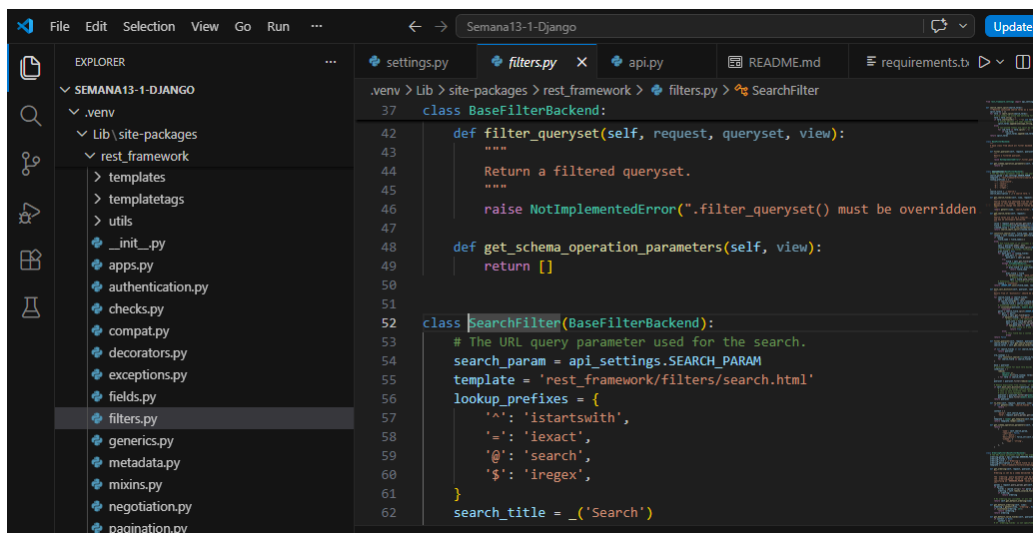
This screenshot shows the implementation of `AuditoriaMiddleware` in `middleware.py`. The code imports `logging` and `time`, and uses `django.utils.deprecation` to import `MiddleWareMixin`. A logger is configured for 'catalogo.audit'. The `__init__` method sets `self.get_response` and logs the initialization. The `__call__` method logs the request time, obtains the client IP, and updates the session with the current URL.

```
20
21 import logging
22 import time
23 from django.utils.deprecation import MiddleWareMixin
24
25 # Logger específico para auditoria (configurado en settings.py)
26 logger = logging.getLogger('catalogo.audit')
27
28
29 class AuditoriaMiddleware:
30
31     def __init__(self, get_response):
32         self.get_response = get_response
33         logger.info("AuditoriaMiddleware inicializado.")
34
35     def __call__(self, request):
36         # -- PROCESAMIENTO DEL REQUEST (antes de la vista) --
37         inicio = time.time()
38
39         # Obtener IP real (considera proxies/load balancers)
40         ip_cliente = self._obtener_ip(request)
41
42         # Guardar última URL visitada en sesión (útil para redirect-afte
43         # request.session es un diccionario especial que persiste entre
```



This screenshot shows the implementation of `SessionInfoMiddleware` in `middleware.py`. It inherits from `AuditoriaMiddleware`. The `__init__` method sets `self.get_response`. The `__call__` method increments the `total_visitas` in the session and registers a `primera_visita` timestamp if it's the first visit. It then calls `self.get_response(request)` and returns the response.

```
29 class AuditoriaMiddleware:
30     def __init__(self, get_response):
31         self.get_response = get_response
32
33     def __call__(self, request):
34         # -- PROCESAMIENTO DEL REQUEST (antes de la vista) --
35         inicio = time.time()
36
37         # Obtener IP real (considera proxies/load balancers)
38         ip_cliente = self._obtener_ip(request)
39
40         # Guardar última URL visitada en sesión (útil para redirect-afte
41         # request.session es un diccionario especial que persiste entre
42
43
44 class SessionInfoMiddleware:
45
46     def __init__(self, get_response):
47         self.get_response = get_response
48
49     def __call__(self, request):
50         # Contar visitas del usuario en esta sesión
51         visitas = request.session.get('total_visitas', 0)
52         request.session['total_visitas'] = visitas + 1
53
54         # Primera visita: registrar timestamp
55         if visitas == 0:
56             from django.utils import timezone
57             request.session['primera_visita'] = timezone.now().isoformat
58
59         response = self.get_response(request)
60         return response
```

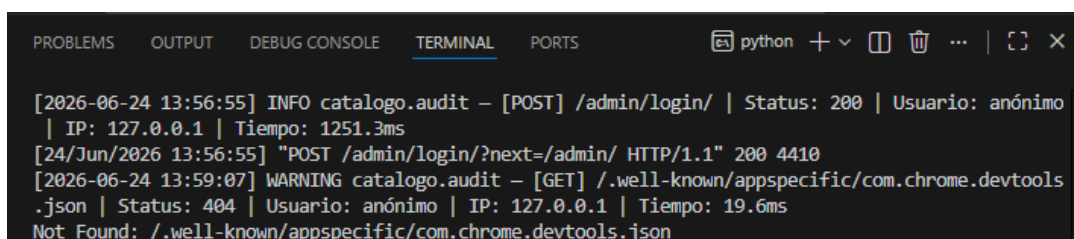
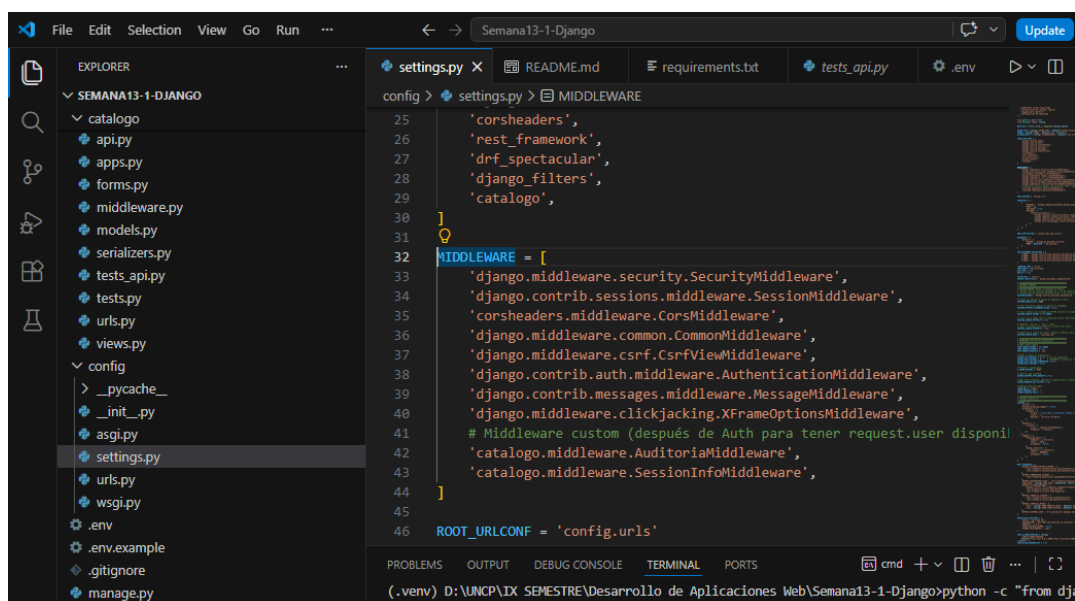
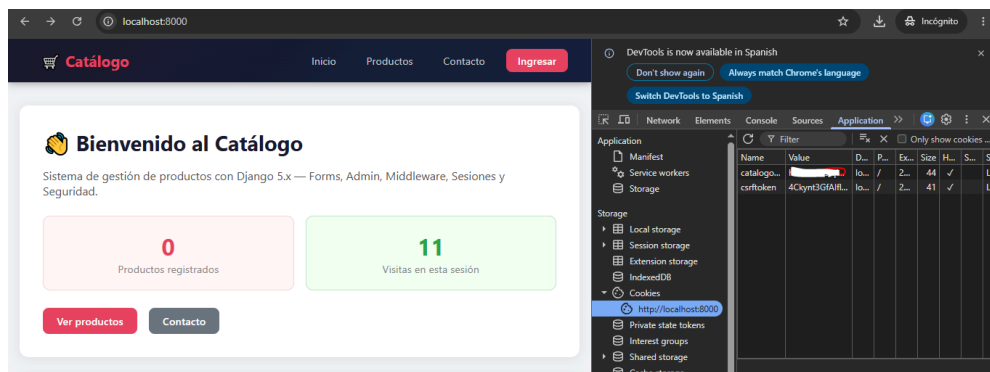


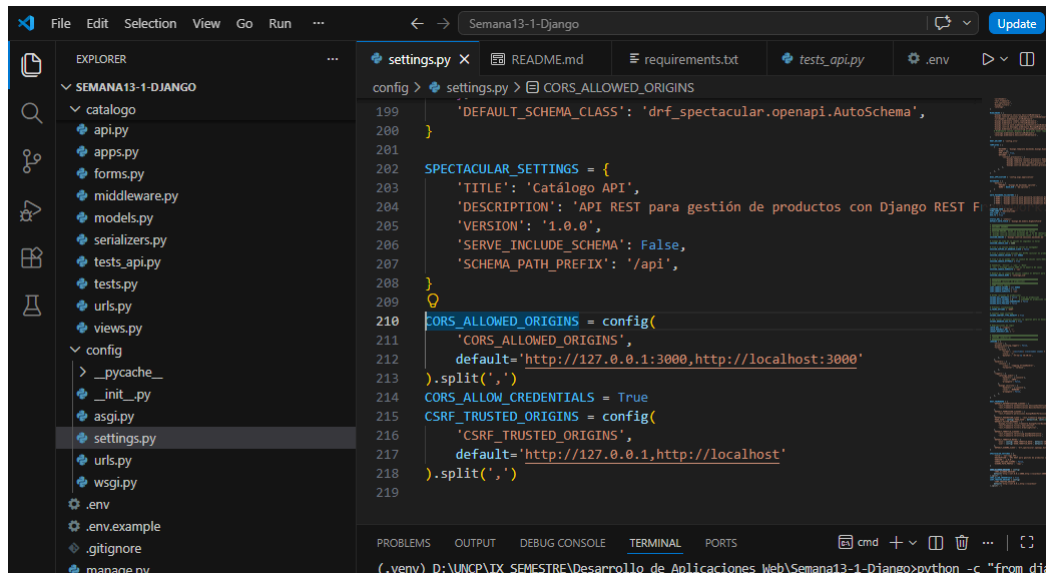
This screenshot shows the implementation of `BaseFilterBackend` and `SearchFilter` in `filters.py`. `BaseFilterBackend` defines `filter_queryset` and `get_schema_operation_parameters`. `SearchFilter` inherits from `BaseFilterBackend` and implements `filter_queryset` to filter by search parameters like `istartswith`, `iexact`, `search`, and `iregex`.

```
37 class BaseFilterBackend:
38
39     def filter_queryset(self, request, queryset, view):
40         """
41         Return a filtered queryset.
42         """
43         raise NotImplementedError(".filter_queryset() must be overridden")
44
45     def get_schema_operation_parameters(self, view):
46         return []
47
48 class SearchFilter(BaseFilterBackend):
49     # The URL query parameter used for the search.
50     search_param = api_settings.SEARCH_PARAM
51     template = 'rest_framework/filters/search.html'
52     lookup_prefixes = {
53         '^': 'istartswith',
54         '=': 'iexact',
55         '@': 'search',
56         '$': 'iregex',
57     }
58     search_title = _('Search')
```

5. CORS, CSRF & Documentación: Configurar `django-cors-headers` middleware y orígenes permitidos. Manejar CSRF para AJAX/SPA (`XCSRFTOKEN` header o

@csrf_exempt solo en endpoints públicos). Generar esquema OpenAPI con drfspectacular y servir en /api/docs/. Probar con Postman/cURL.

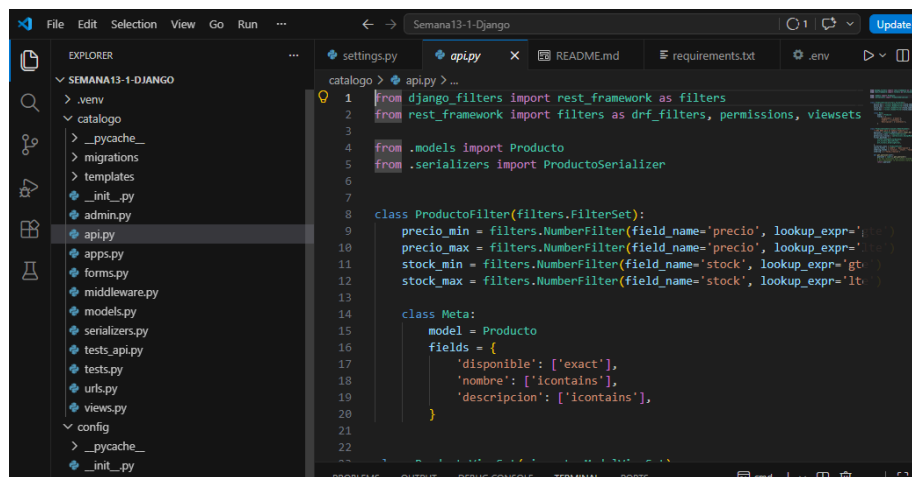




```
config > settings.py > CORS_ALLOWED_ORIGINS
199 'DEFAULT_SCHEMA_CLASS': 'drf_spectacular.openapi.AutoSchema',
200 }
201
202 SPECTACULAR_SETTINGS = {
203     'TITLE': 'Catálogo API',
204     'DESCRIPTION': 'API REST para gestión de productos con Django REST F
205     'VERSION': '1.0.0',
206     'SERVE_INCLUDE_SCHEMA': False,
207     'SCHEMA_PATH_PREFIX': '/api',
208 }
209
210 CORS_ALLOWED_ORIGINS = config(
211     'CORS_ALLOWED_ORIGINS',
212     default='http://127.0.0.1:3000,http://localhost:3000'
213 ).split(',')
214
215 CORS_ALLOW_CREDENTIALS = True
216
217 CSRF_TRUSTED_ORIGINS = config(
218     'CSRF_TRUSTED_ORIGINS',
219     default='http://127.0.0.1,http://localhost'
220 ).split(',')
221
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

(.venv) D:\UNCPVX-SEMESTRE\Desarrollo de Aplicaciones Web\Semana13-1-Django>python -c "from di



```
catalogo > api.py > ...
1 from django_filters import rest_framework as filters
2 from rest_framework import filters as drf_filters, permissions, viewsets
3
4 from .models import Producto
5 from .serializers import ProductoSerializer
6
7
8 class ProductFilter(filters.FilterSet):
9     precio_min = filters.NumberFilter(field_name='precio', lookup_expr='gte')
10    precio_max = filters.NumberFilter(field_name='precio', lookup_expr='lte')
11    stock_min = filters.NumberFilter(field_name='stock', lookup_expr='gte')
12    stock_max = filters.NumberFilter(field_name='stock', lookup_expr='lte')
13
14    class Meta:
15        model = Producto
16        fields = {
17            'disponible': ['exact'],
18            'nombre': ['icontains'],
19            'descripcion': ['icontains'],
20        }
21
22
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Link en GitHub: <https://github.com/GErickOBTS/Semana13-1-Django>